

# Sri Lanka Institute of Information Technology



Lab Submission  
<Lab sheet 07>

<IT25103681>

<Hasaranga S.O.>

**Discrete Mathematics | IT1160**

B.Sc. (Hons) in Information Technology

## Part 01

01)

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
```

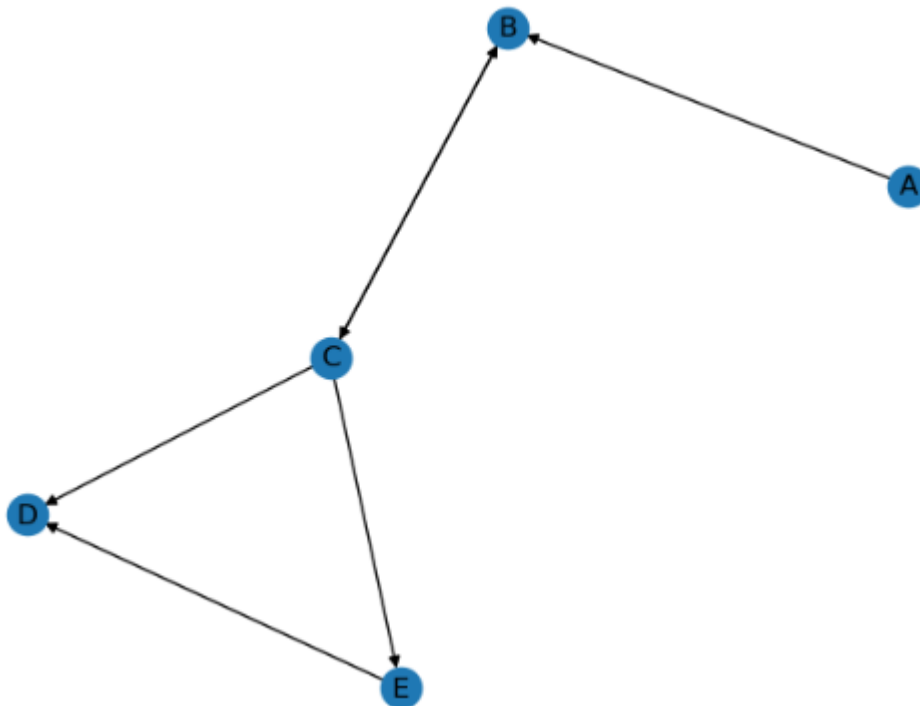
```
#que01-I
G=nx.DiGraph()

G.add_nodes_from(["A","B","C","D","E"])
G.add_edges_from([("A","B"),("B","C"),("C","D"),("C","E"),("E","D"),("C","B")])
adj_list=nx.to_dict_of_lists(G)
print(adj_list)
```

```
matrix=nx.to_numpy_array(G)
print(matrix)
```

```
{'A': ['B'], 'B': ['C'], 'C': ['D', 'E', 'B'], 'D': [], 'E': ['D']}
[[0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 1. 0. 1. 1.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0.]]
```

```
#que01-II
nx.draw(G, with_labels = True)
plt.show()
```



02)

```
#que02
print(G.in_degree())
print(G.out_degree())
print(list(nx.selfloop_edges(G)))
```

```
[('A', 0), ('B', 2), ('C', 1), ('D', 2), ('E', 1)]
[('A', 1), ('B', 1), ('C', 3), ('D', 0), ('E', 1)]
[]
```

03)

```
#que03-I
if nx.has_path(G, "A", "D"):
    print(list(nx.all_simple_paths(G, source="A", target="D")))
```

```
[['A', 'B', 'C', 'D'], ['A', 'B', 'C', 'E', 'D']]
```

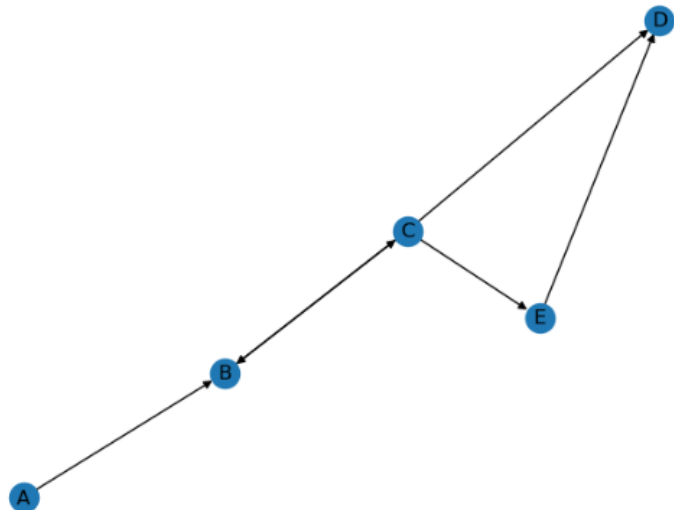
```
#que03-II
print(list(nx.simple_cycles(G)))
```

```
[['C', 'B']]
```

04)

```
#que04-I
T=nx.DiGraph()

T.add_nodes_from([1,2,3,4,5,6,7,8])
T.add_edges_from([(1,2),(1,3),(2,4),(4,2),(2,5),(3,5),(5,3),(4,6),(5,6),(6,7),(7,8),(8,6)])
nx.draw(G, with_labels = True)
plt.show()
```



```
#que04-II
adj_list=nx.to_dict_of_lists(T)
print(adj_list)
```

```
matrix=nx.to_numpy_array(T)
print(matrix)
```

```
{1: [2, 3], 2: [4, 5], 3: [5], 4: [2, 6], 5: [3, 6], 6: [7], 7: [8], 8: [6]}
[[0. 1. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 1. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0. 1. 0. 0.]
 [0. 0. 0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 0. 0. 0. 1.]
 [0. 0. 0. 0. 0. 1. 0. 0.]]
```

05)

```
#que05-1
print(list(nx.simple_cycles(T)))
```

```
[[2, 4], [3, 5], [8, 6, 7]]
```

```
#que05-II
T.remove_edge(4,2)
T.remove_edge(3,5)
T.remove_edge(8,6)
print(list(nx.simple_cycles(T)))
```

```
[]
```

```
#que05-III
for nodes in range (1,8):
    if nx.has_path(T, nodes,8):
        path=nx.shortest_path(T,nodes,8)
        print(nodes,"->8:",path)
    else:
        print(nodes,"->8: No Path")
```

```
1 ->8: [1, 2, 4, 6, 7, 8]
```

```
2 ->8: [2, 4, 6, 7, 8]
```

```
3 ->8: No Path
```

```
4 ->8: [4, 6, 7, 8]
```

```
5 ->8: [5, 6, 7, 8]
```

```
6 ->8: [6, 7, 8]
```

```
7 ->8: [7, 8]
```

## Part 02

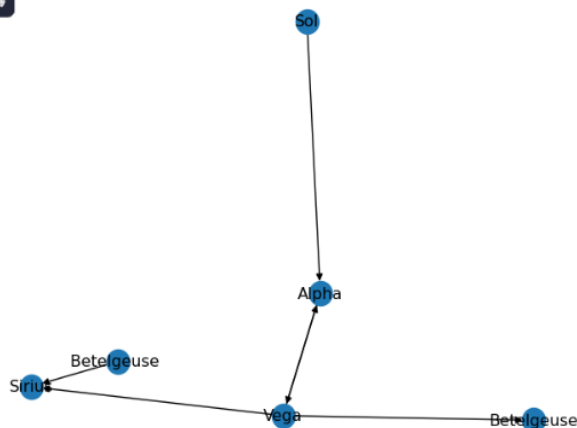
01)

```
#que01-I
S=nx.DiGraph()
S.add_nodes_from(["Sol", "Alpha", "Vega", "Sirius", "Betelgeuse"])
S.add_edges_from([("Sol", "Alpha"), ("Alpha", "Vega"), ("Vega", "Sirius"), ("Vega", "Betelgeuse"), ("Betelgeuse", "Sirius"), ("Vega", "Alpha")])
```

```
#que01-II
adj_list=nx.to_dict_of_lists(S)
print(adj_list)

matrix=nx.to_numpy_array(S)
print(matrix)
```

```
{'sol': ['Alpha'], 'Alpha': ['Vega'], 'Vega': ['Sirius', 'Betelgeuse', 'Alpha'], 'Sirius': [], 'Betelgeuse': [], 'Betelgeuse ': ['Sirius']}
[[0. 1. 0. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0.]
 [0. 1. 0. 1. 1. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0.]]
```



02)

```
#que02
print(S.in_degree())
print(S.out_degree())
print(list(nx.selfloop_edges(S)))

[('Sol', 0), ('Alpha', 2), ('Vega', 1), ('Sirius', 2), ('Betelgeuse', 1), ('Betelgeuse ', 0)]
[('Sol', 1), ('Alpha', 1), ('Vega', 3), ('Sirius', 0), ('Betelgeuse', 0), ('Betelgeuse ', 1)]
[]
```

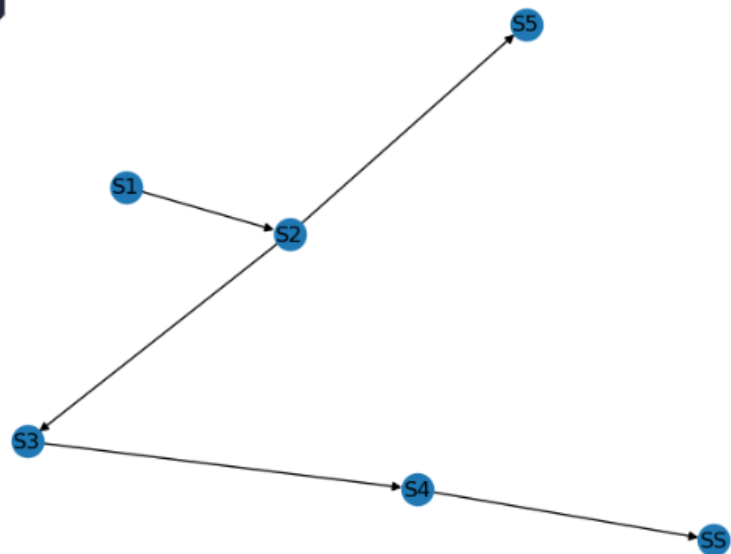
03)

```
#que03
print(list(nx.all_simple_paths(S, source="Sol", target="Sirius")))
print(list(nx.simple_cycles(S)))

[['Sol', 'Alpha', 'Vega', 'Sirius']]
[['Vega', 'Alpha']]
```

04)

```
#que04-I
N=nx.DiGraph()
N.add_nodes_from(["S1", "S2", "S3", "S4", "S5"])
N.add_edges_from([("S1","S2"),("S2","S3"),("S3","S4"),("S4","S5"),("S2","S5")])
nx.draw(N, with_labels = True)
plt.show()
```



```
#que04-II
matrix=nx.to_numpy_array(N)
print(matrix)
```

```
[[0. 1. 0. 0. 0. 0.]
 [0. 0. 1. 0. 1. 0.]
 [0. 0. 0. 1. 0. 0.]
 [0. 0. 0. 0. 0. 1.]
 [0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0.]]
```

05)

```
#que05
print(N.in_degree())
print(N.out_degree())
print(nx.has_path(N,"S1","S5"))
print(nx.shortest_path(N,"S1","S5"))

[('S1', 0), ('S2', 1), ('S3', 1), ('S4', 1), ('S5', 1), ('SS', 1)]
[('S1', 1), ('S2', 2), ('S3', 1), ('S4', 1), ('S5', 0), ('SS', 0)]
True
['S1', 'S2', 'S5']
```

---